Types of constructor

① **Default Constructor**

It takes no arguements.

Ex
```
#include <iostream>
using namespace std;
class student {
    public:
        student () {
            cout << "Default Constructor Called\n";
        }
};
int main () {
    student s1;
}
```

② **Parameterized Constructor.**

- Takes arguement to initialized values.

```
#include <iostream>
using namespace std;
class student {
    public :
        int id ;
        student (int i) {
            id = i
        }
        void display () {
            cout << "ID:" << id ;
        }
};
int main () {
    student s1 (101) ;
    s1. display ();
}
```

Problem statements: Case class initialization using constructor

What actually happen ?

- instead of
  add (3,4);
- compiler may replace it with: 3+4

Pros
- Faster execution
- usefull for small and simple function

Cons
- Not suitable for large function.
- Can increase code size
- compiler may ignore inline sometimes.

Important points
- inline is just a request not gurentee
- Compiler decide wather to inline or not.

Problem statement : Simple calculator using inline function

Day 20 C constructors

A constructor is a special member function of a class that is automatically called when an object is created

It main porpose:
- Initialize object data
- set default values
- Allocate resources if needed

Key features
- Same name as class
- No returntype (not even void)
- called automatically
- Can be overload

Syntax:

```
class Classname {
    public :
        Classname () {
            // Constructor code
        }
};
```

Point

- friend function breaks encapsulation.
- It must be declared in all classes whose private data it access.
- It can be a global function or even a member of another class.

Real life Analogy

Think of a class as a private room, only member can enter.
A friend function = special guest without permission.

Problem statement: Sum of private data using friend fn

- two classes
- int a of first and b of 2nd
- add both a and b.

Day-19 (Inline function)

An inline function is a function where the compiler tries to replace the function call with the actual code of the function.

Syntax: inline return_type function_name (parameters) {
    // code
}

Example:

```
#include <iostream>
using namespace std;
inline int add( int a, int b) {
    return a+b ;
}
int main () {
Cout << add(3,7) ;
return 0; }
```

How it works?

- Same constructor name (same as class)
- Different Parameter list.
- Compiler decide which one to calls base on argum

Key Point
- constructor can be overload like function
- Must differ in number ~~of~~ or type of param
- Return type does not matter.

Problem statement : ~~class~~ Volume Box using constructor ov

Day 21 (Constructor overloading)

Constructor overloading means having multiple constructor in the same class with different parameters

It allows to be initialized in different ways

Why do we use it?

- Flexibility in object creation
- Initialization object with different inputs
- Improve readability

Example:

```
#include<string>
#include<iostream>
using namespace std;
class student {
    public:
        int id;
        string name;

    // default constructor
    student () {
        id = 0;
        name = "unknown";
    }

    // Parameterized constructor
    student (int i, string n) {
        id = i;
        name = n;
    }

    void display () {
        cout << id << " " << name << endl;
    }
};

int main {
    student s1;   // calls default
    student s2 (101, "Kaush");   // calls parameterized
    s1.display;
}
```